

PySpark Theoretical Assignment 11

ACID Properties, Delta Lake, Schema Evolution & Table Maintenance

Introduction: PySpark and Delta Lake

PySpark is the Python API for Apache Spark; it is not itself a storage format or transactional table layer. When we use ordinary files such as CSV or Parquet directly, we do not automatically get all the transactional and table-management features that Delta Lake provides.

Delta Lake adds a table-management layer on top of data files. It stores data in Parquet files and maintains a transaction log in the `_delta_log` directory. The log records committed changes and identifies the files belonging to each table version.

1. Explain the ACID properties

Atomicity — A transaction is one complete unit: either all changes are committed, or none are. An incomplete write should not appear as a successful table update.

Consistency — A transaction preserves defined rules and integrity. Delta schema enforcement helps prevent incompatible writes.

Isolation — Concurrent transactions should not expose partially committed changes. Readers see a consistent committed table snapshot.

Durability — Once committed successfully, changes are recorded persistently and remain available after the Spark application stops.

How Delta supports ACID: Delta Lake combines Parquet data files with a transaction log to manage transactions and consistent table snapshots.

2. What are the shortcomings of PySpark? Why do we need Delta Lake?

Your answer is partially correct, but these are more accurately limitations of using ordinary files without an additional transactional table layer.

Your point	Correction
Doesn't support ACID	Ordinary file writes do not automatically provide Delta-style table transactions.
Doesn't support schema changes	PySpark can handle schema changes, but ordinary file workflows do not automatically provide controlled, transac
Doesn't support modifying history	Ordinary files do not automatically maintain Delta-style version history or time travel.

Delta Lake adds ACID transactions, schema enforcement and evolution, time travel, table history, UPDATE/DELETE/MERGE operations, and maintenance operations such as OPTIMIZE and VACUUM.

Key takeaway: We do not replace PySpark with Delta Lake. We use Delta Lake with Spark/PySpark to make data storage and table operations more reliable.

3. Why do we use mergeSchema = "true" in PySpark?

mergeSchema enables supported schema evolution during a Delta write. It allows new columns in the incoming DataFrame to be added to the existing Delta table schema. Without schema evolution, an append that introduces additional columns will generally fail schema validation.

Iteration 1 — Create the initial table

```
data = [(1, 'charan'), (2, 'hari')]
df = spark.createDataFrame(data, ['id', 'name'])
df.write.mode('overwrite').format('delta').save('details_delta')
```

Initial schema: id (long), name (string).

Iteration 2 — Append a new column

```
data = [(3, 'rahul', 'IT'), (4, 'ram', 'HR')]
df = spark.createDataFrame(data, ['id', 'name', 'dept'])
df.write.mode('append').option('mergeSchema', 'true').format('delta').save('details_delta')
```

The incoming DataFrame introduces dept. With mergeSchema enabled, the table schema can evolve to id, name, dept. Existing rows have NULL for dept because the column did not exist when they were written.

Iteration 3: Incoming columns are id and name. Existing dept remains; new rows have NULL for dept.

Iteration 4: Incoming columns are id and dept. Existing name remains; new rows have NULL for name.

Iteration 5: Incoming columns are dept and HR. This adds a column literally named HR; it is not a special keyword here.

Iteration 6: Incoming column is city. Delta adds city; earlier rows have NULL for city.

Final schema: id, name, dept, HR, city. The final table contains 12 rows if all six two-row writes succeed.

Important: mergeSchema adds supported new columns. It is not a general-purpose command for renaming columns, removing columns, or arbitrarily changing existing data types.

4. How do you read and write Delta tables in PySpark?

A. Write / overwrite a Delta table

```
df.write.format('delta').mode('overwrite').save('details_delta')
```

format('delta') specifies Delta format. mode('overwrite') replaces existing table data at that path.

B. Append to an existing Delta table

```
df.write.format('delta').mode('append').save('details_delta')
```

Append adds incoming records. For supported new-column schema evolution:

```
df.write.format('delta').mode('append')
.option('mergeSchema', 'true')
.save('details_delta')
```

C. Read a Delta table

```
details_df = spark.read.format('delta').load('details_delta')
details_df.show()
details_df.printSchema()
```

D. Read a specific version (time travel)

```
version3_df = spark.read.format('delta')
.option('versionAsOf', 3)
.load('details_delta')
version3_df.show()
```

This reads version 3, provided the required historical data files are still available.

E. Read a table registered in the Spark catalog

```
df = spark.read.format('delta').table('details_table')
```

5. Explain VACUUM in Delta Lake with an example

VACUUM removes obsolete data files no longer needed by the current Delta table, subject to the retention period.

UPDATE, DELETE, MERGE, or compaction can make older files obsolete; those files may be retained for time travel.

```
Version 0 → part-0001.parquet
After an update, Version 1 uses part-0002.parquet.
part-0001.parquet may be obsolete but retained for older-version reads.
```

Example

```
detailsTable = DeltaTable.forPath(spark, 'details_delta')
detailsTable.vacuum(168)
```

168 hours equals 7 days. VACUUM removes eligible obsolete files older than the retention period.

What does vacuum(0) mean? It requests zero-hour retention. It can delete obsolete files without the normal waiting period if the runtime configuration permits it.

Your class configuration sets spark.databricks.delta.retentionDurationCheck.enabled to false, disabling the retention safety check. **Warning:** vacuum(0) can permanently remove files needed for time travel or restoring older versions, and can affect readers or streaming jobs that still rely on them. Avoid it casually in production.

VACUUM normally does not delete current table data. It deletes eligible obsolete data files. It is not the same as directly deleting the transaction log, although removing old data files can make old versions unreadable.

6. Difference between OPTIMIZE and VACUUM

A. OPTIMIZE

OPTIMIZE improves data layout by compacting smaller files into larger files. This can reduce the number of files Spark processes and improve query performance.

```
detailsTable.optimize().executeCompaction()
```

Before: small-1.parquet, small-2.parquet, small-3.parquet → After: a larger compacted file. Old files may become obsolete but are not necessarily deleted immediately.

B. VACUUM

VACUUM removes eligible obsolete files after the configured retention period.

```
detailsTable.vacuum(168)
```

Feature	OPTIMIZE	VACUUM
Main purpose	Improve query performance	Clean up obsolete files
Primary operation	Compacts small files	Physically deletes eligible old files
Improves file layout	Yes	No
Frees storage	Not necessarily immediately	Yes, by deleting eligible files
Time travel impact	Old files may remain available	Can remove files needed for older versions

Can we use both?

```
detailsTable.optimize().executeCompaction()  
detailsTable.vacuum(168)
```

OPTIMIZE improves file layout; VACUUM cleans up eligible obsolete files. Choose retention with time travel, concurrent jobs, and streaming workloads in mind.

7. How do history(), time travel, and restoreToVersion() work?

A. View table history

```
detailsTable = DeltaTable.forPath(spark, 'details_delta')
detailsTable.history().show(truncate=False)
```

History shows recorded table operations, including operation type, version, timestamp, and available operation details.

B. Read an earlier version

```
version3_df = spark.read.format('delta')
  .option('versionAsOf', 3)
  .load('details_delta')
```

C. Restore an earlier version

```
detailsTable.restoreToVersion(3)
```

This creates a new table operation that restores the table to the state represented by version 3. It does not erase all later history; the restore itself is recorded as a new operation. Restoration may fail if required historical data files have already been removed by VACUUM.

Final revision: remember these differences

Concept	Remember
ACID	Atomicity, Consistency, Isolation, Durability
Delta Lake	Transactional table layer on top of data files
mergeSchema	Allows supported schema evolution, such as adding columns
format('delta')	Reads or writes data in Delta format
history()	Inspects recorded table operations
versionAsOf	Reads a previous table version
restoreToVersion()	Restores a previous state through a new operation
OPTIMIZE	Compacts files to improve query performance
VACUUM	Removes eligible obsolete files

One-line memory trick

mergeSchema → Evolve the schema.

history() → Inspect the changes.

versionAsOf → Read the past.

restoreToVersion() → Restore a past state.

OPTIMIZE → Compact files.

VACUUM → Remove obsolete files.